FPT UNIVERSITY | EMBEDDED LINUX SYSTEMS

BÁO CÁO THỰC HÀNH
LAB–01

CẤU HÌNH KERNEL & CHUỖI KHỞI ĐỘNG HỆ THỐNG NHÚNG

ARMv7-A Cortex-A9 · QEMU vexpress-a9 · Linux 5.15.0

NHÓM 05 • DANH SÁCH THÀNH VIÊN

HỌ VÀ TÊN	MSSV
Lê Tiến Thành	SE201799
Lê Tấn Lộc	SE201412
Nguyễn Quang Huy Đạt	SE201415
Trần Đình Khôi Nguyên	SE200558
Trần Trương Tuấn Vũ	SE196467

Phạm vi báo cáo: Kernel / U-Boot / initramfs / BusyBox / QEMU

Dữ liệu thực nghiệm của nhóm · Tổng hợp: 21/10/2026

TÓM TẮT & PHẠM VI THỰC NGHIỆM

Báo cáo ghi lại việc cấu hình nhân Linux 5.15 cho ARMv7, khởi động U-Boot trên máy ảo Versatile Express, lắp ghép root filesystem từ BusyBox và kiểm tra hành vi hệ thống sau boot. Các ảnh console lấy từ nhật ký thực hành của nhóm; phần kiểm thử bổ sung tháng 10 dùng bản clone repository LOS201. Các nhận định đều gắn với bằng chứng từ cấu hình, ảnh terminal và kết quả boot đã ghi nhận.

Bản đồ nội dung

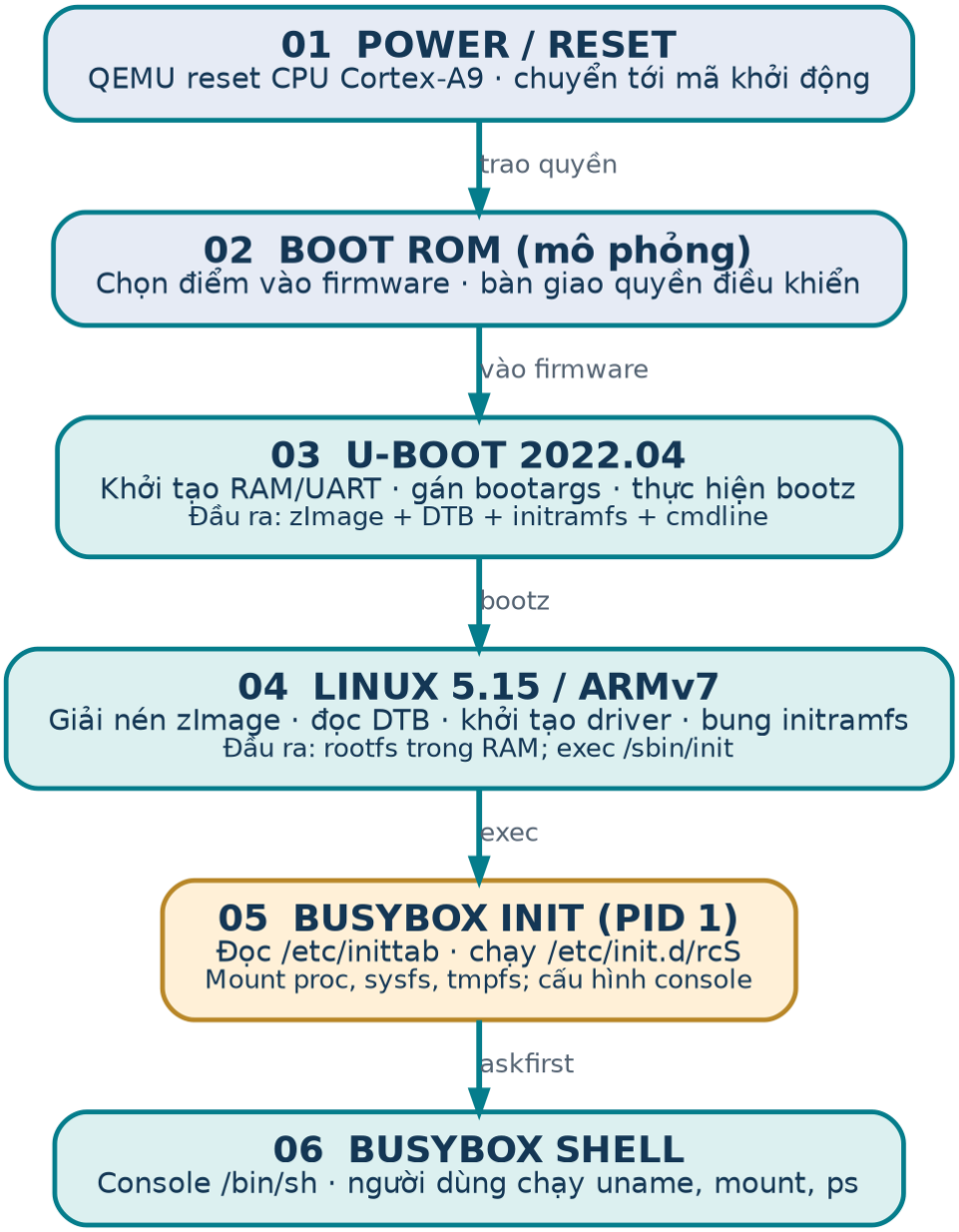
PHẦN	NỘI DUNG
01	Nền tảng và tiêu chí kiểm thử
02	Kiến trúc và đường đi khởi động
03	Phần A — Build Kernel ARM
04	Phần B — U-Boot và bootz
05	Phần C — Initramfs / BusyBox
06	Giải thích 5 câu phân tích
07	Khó khăn / kết luận
08	Phụ lục — Cách tái lập kiểm thử

1. Nền tảng kiểm thử và kết quả mục tiêu

Thành phần	Thiết lập được ghi nhận
Target	QEMU vexpress-a9, ARM Cortex-A9, 512 MiB RAM
Nhân Linux	5.15.0; zImage ARM, DTB vexpress-v2p-ca9.dtb
Firmware	U-Boot 2022.04; console serial
Rootfs	BusyBox 1.35.0, binary ARM EABI5 liên kết tĩnh
Cross compiler	arm-linux-gnueabihf-; GCC 13.3 được ghi trong boot log mới
Bài test chính	U-Boot → bootz → Linux → /sbin/init → /bin/sh

Điều kiện hoàn thành chức năng là shell tương tác được, uname báo armv7l, PID 1 là init và proc/sysfs được mount. Việc xuất hiện chữ “SMP” trong uname chứng tỏ nhân được biên dịch có SMP, không chứng minh máy ảo đang chạy đúng số lõi đã yêu cầu.

2. Kiến trúc và đường đi của quá trình boot



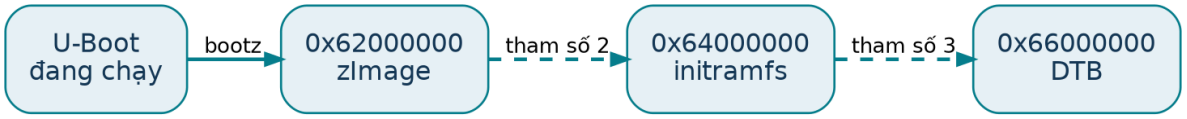
Hình 1. Sơ đồ luồng khởi động được thiết kế lại: tác nhân thực thi, công việc và dữ liệu bàn giao.

2.1. Phân biệt các công đoạn

QEMU thay thế phần cứng thật và chọn điểm nạp firmware. U-Boot chuẩn bị môi trường tối thiểu, cung cấp tham số dòng lệnh và địa chỉ các image cho bootz. Nhân Linux nhận thông tin phần cứng từ DTB, giải nén initramfs và tạo môi trường có thể thực thi /sbin/init. Từ đây BusyBox init điều phối /etc/inittab, rcS và mở console shell. Trong lần boot trực tiếp bằng -kernel zImage, giai đoạn U-Boot được bỏ qua; lần thử tích hợp lại xác nhận giao tiếp U-Boot → Kernel.

2.2. Địa chỉ nạp image và vai trò của DTB

Đề hướng dẫn bộ địa chỉ 0x60100000 / 0x64000000 / 0x60000000. Trong thử nghiệm bổ sung, các tệp được đặt tại 0x62000000 (zImage), 0x64000000 (initramfs) và 0x66000000 (DTB) nhằm tránh ghi đè vùng mà U-Boot cần dùng khi chuẩn bị FDT. Đây là bố trí để kiểm thử thực tế, không nên nhầm với cấu hình mặc định của đề.



Hình 2. Bộ ba địa chỉ thực nghiệm thành công trong không gian RAM của QEMU.

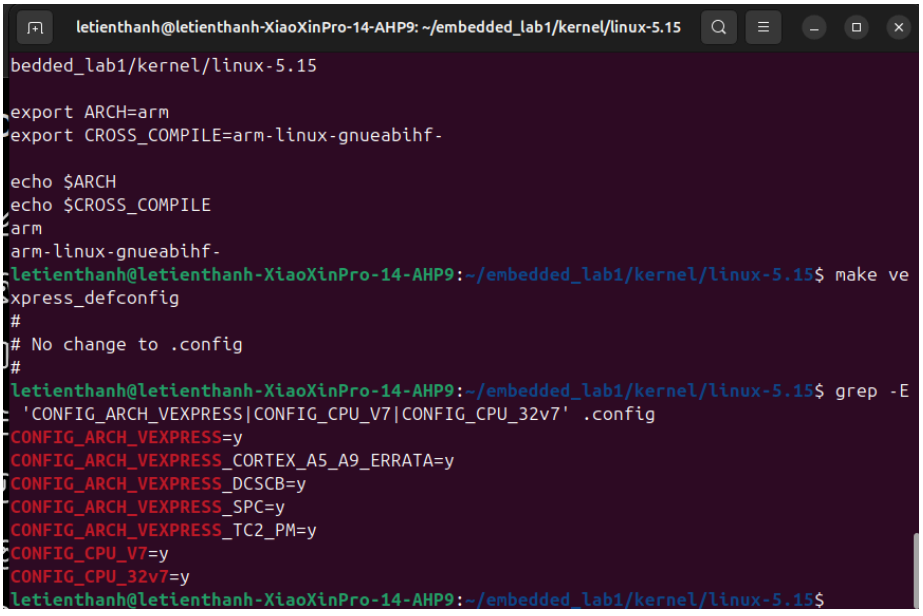
DTB có magic 0xd00dfeed và tổng kích thước 0x3701. Ở lần thử dùng DTB tại 0x60000000, bootz báo FDT_ERR_BADSTRUCTURE, nên nhóm đổi vị trí nạp rồi xác minh bằng fdt header và fdt print /chosen. Sau đó U-Boot đã hiển thị Starting kernel và chuyển sang Linux. Kích thước initramfs trong tham số bootz phải là giá trị hex thực lấy từ stat, không phải chuỗi giữ chỗ.

3. Phần A — Biên dịch Kernel cho ARMv7

3.1. Chọn cấu hình đúng board

Quá trình cấu hình đặt ARCH=arm và CROSS_COMPILE=arm-linux-gnueabihf-. Nhóm sử dụng vexpress_defconfig cho Cortex-A9; versatile_defconfig là cấu hình khác nền tảng. Các lựa chọn then chốt được kiểm tra là CONFIG_ARCH_VEXPRESS, CONFIG_CPU_V7, CONFIG_BLK_DEV_INITRD, PL011 UART console và CONFIG_SMP.

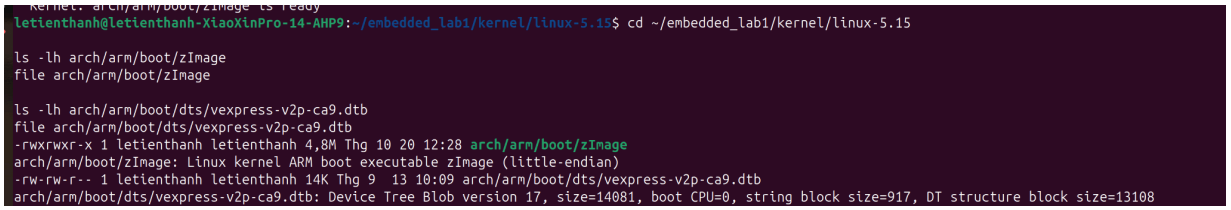
```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
make vexpress_defconfig
make menuconfig
make -j$(nproc) zImage dtbs modules
```



Hình 3. Minh chứng của nhóm: cấu hình vexpress và các cờ ARMv7 trong .config.

3.2. Kết quả build và kiểm tra binary

Log thực hành ghi nhận mục tiêu zImage và Device Tree được tạo. Trong bản clone GitHub, lệnh file xác nhận output/zImage là Linux kernel ARM boot executable (little-endian). Khi boot, uname -a báo Linux 5.15.0 armv7l, qua đó đối chiếu được binary trên repository với phần mềm thật sự chạy.



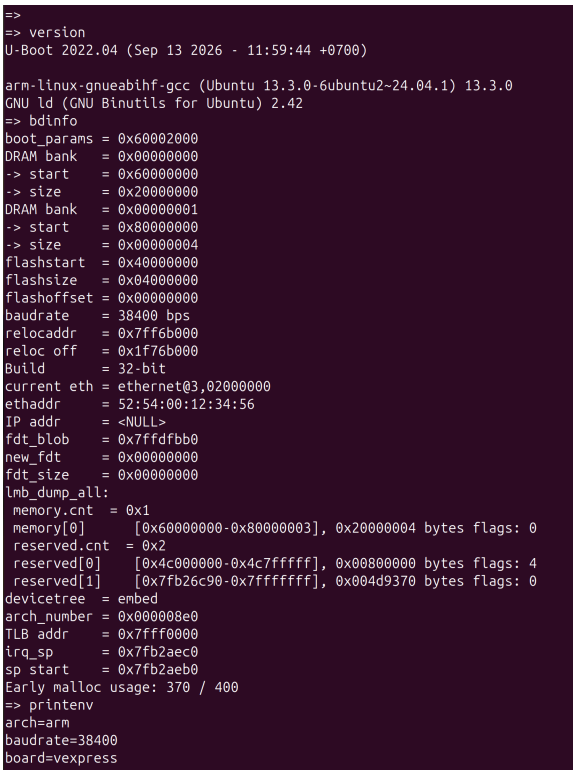
Hình 4. Kiểm tra zImage và DTB sau khi biên dịch trong thư mục source của nhóm.

Dấu hiệu	Ý nghĩa kiểm chứng
CONFIG_ARCH_VEXPRESS=y	Nhân có hỗ trợ nền tảng Versatile Express
CONFIG_BLK_DEV_INITRD=y	Hỗ trợ initramfs/initrd
CONFIG_SERIAL_AMBA_PL011=y	Có driver UART PL011
file zImage: ARM boot executable	Đúng định dạng Kernel ARM
uname: 5.15.0 armv7l	Kernel ARM đã thực sự chạy

4. Phần B — U-Boot, bootargs và nạp Kernel

4.1. Môi trường U-Boot và lệnh shell

Tại máy ảo, U-Boot 2022.04 nhận 512 MiB DRAM, 64 MiB Flash, UART serial và ethernet. Cảnh báo bad CRC nghĩa là không có environment hợp lệ trong Flash giả lập nên chương trình dùng environment mặc định; bootloader vẫn vào được dấu nhắc =>. Các lệnh version, bdinfo, printenv, setenv và help bootz đã được thao tác.



Hình 5. Log U-Boot từ bài thực hành: version, RAM và thông tin board.

printenv bootargs
printenv bootcmd
setenv bootargs 'console=ttyAMA0,115200 rdinit=/sbin/init'
printenv bootargs

```
help bootz
```

bootargs là chuỗi Linux nhận tại /proc/cmdline; bootcmd là tập lệnh U-Boot chạy khi được gọi. Khi chưa thay đổi, bootcmd mặc định chạy distro_bootcmd/bootflash, có thể thử MMC hoặc TFTP; đây không phải phép thử nạp initramfs của nhóm.

4.2. Chẩn đoán boot lỗi do thiếu rootfs

Khi chưa cung cấp root filesystem phù hợp, Kernel có thể khởi tạo phần cứng rồi dừng ở bước mount root và in thông báo VFS panic. Bài thực hành ban đầu đã lưu màn hình lỗi này. Đây là đối chứng âm cho thấy Kernel chạy được chưa đủ; còn cần initramfs và init hợp lệ.

```
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
sii902x 0-0039: supply iovcc not found, using dummy regulator
sii902x 0-0039: supply cvcc12 not found, using dummy regulator
i2c i2c-0: Added multiplexed i2c bus 2
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: found bridge on endpoint 0
drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
  #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7,
00000000/10007000.kmi/serio1/input/input2
VFS: Cannot open root device "ram0" or unknown-block(0,0): error -6
Please append a correct "root=" boot option; here are the available partitions:
1f00          131072 mtdblock0
 (driver?)
1f01           32768 mtdblock1
 (driver?)
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1
Hardware name: ARM-Versatile Express
[<8010f4c4>] (unwind_backtrace) from [<8010b2dc>] (show_stack+0x10/0x14)
[<8010b2dc>] (show_stack) from [<8084b4ac>] (dump_stack_lvl+0x40/0x4c)
[<8084b4ac>] (dump_stack_lvl) from [<808473b0>] (panic+0xfc/0x2e8)
[<808473b0>] (panic) from [<80b01748>] (mount_block_root+0x1d0/0x274)
[<80b01748>] (mount_block_root) from [<80b01bd8>] (prepare_namespace+0x150/0x18c)
[<80b01bd8>] (prepare_namespace) from [<8084fcbc>] (kernel_init+0x10/0x124)
[<8084fcbc>] (kernel_init) from [<80100130>] (ret_from_fork+0x14/0x24)
Exception stack(0x810a7fb0 to 0x810a7ff8)
7fa0:                                     00000000 00000000 00000000 00000000
7fc0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fe0: 00000000 00000000 00000000 00000000 00000013 00000000
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ]---
```

Hình 6. Trường hợp Kernel panic thiếu rootfs được nhóm ghi lại.

4.3. Kiểm chứng tích hợp U-Boot → Kernel → BusyBox

Ngày 21/10/2026, nhóm clone lại LOS201, nạp sẵn zImage/DTB/initramfs bằng QEMU loader rồi chạy bootz. Sau khi thay đổi vị trí DTB sang 0x66000000, U-Boot báo nạp Ramdisk và Device Tree thành công, sau đó in Starting kernel. Kernel giải nén initramfs và chuyển điều khiển tới /sbin/init; màn hình cuối xuất hiện BusyBox shell. Kết quả này mạnh hơn phép thử chỉ mở U-Boot prompt hoặc boot trực tiếp -kernel zImage.

```
fdt addr 0x66000000
fdt header            # magic 0xd00dfeed; totalsize 0x3701
setenv bootargs 'console=ttyAMA0,115200 rdinit=/sbin/init'
setenv bootcmd 'bootz 0x62000000 0x64000000:<hex-size> 0x66000000'
run bootcmd
```

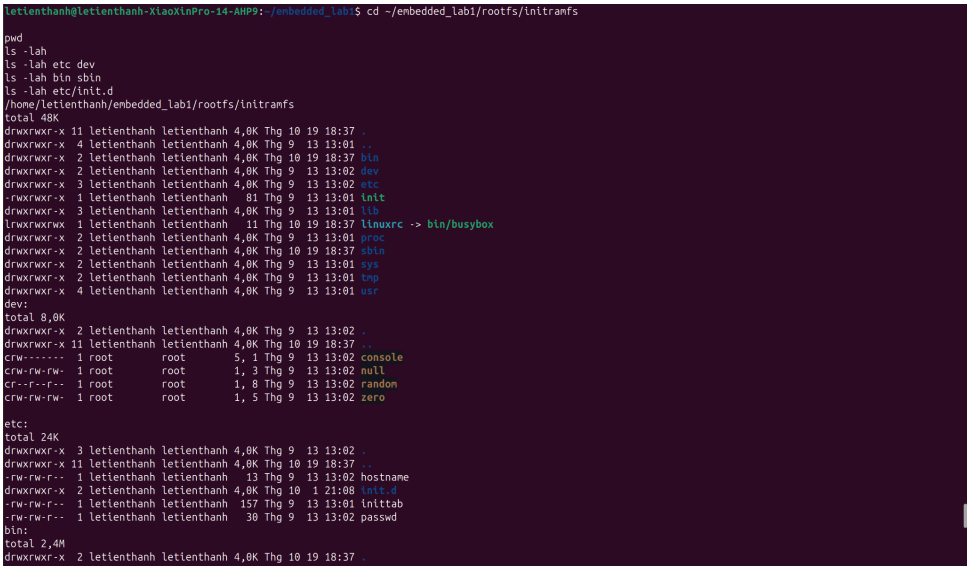
Ký hiệu <hex-size> phải được thay bằng kích thước thực của output/initramfs.cpio.gz tính trên HOST. Trong log do nhóm cung cấp, giá trị 0x1a0000 được dùng và hệ thống có thể boot; cần kiểm tra lại với stat để xác nhận không phụ thuộc việc nạp thừa vùng RAM.

Log từ lần boot tích hợp	Kết quả
Loading Ramdisk ... OK / Loading Device Tree ... OK	U-Boot chuẩn bị thông tin boot
Starting kernel ...	Đã chuyển từ bootloader tới Linux
Unpacking initramfs...	Root filesystem được bung vào RAM
Run /sbin/init as init process	Linux chuyển sang userspace
Boot completed! ... / #	BusyBox shell sẵn sàng

5. Phần C — BusyBox và root filesystem tối giản

5.1. Chuẩn bị initramfs

BusyBox của nhóm là ARM EABI5 liên kết tĩnh. rootfs/initramfs chứa thư mục bin, sbin, etc, dev, proc, sys, tmp...; các device node console, null, zero, random được tạo trước khi đóng gói. Rootfs được nén theo định dạng cpio newc và gzip thành output/initramfs.cpio.gz.



Hình 7. Cấu trúc rootfs/initramfs từ dữ liệu bài thực hành của nhóm.

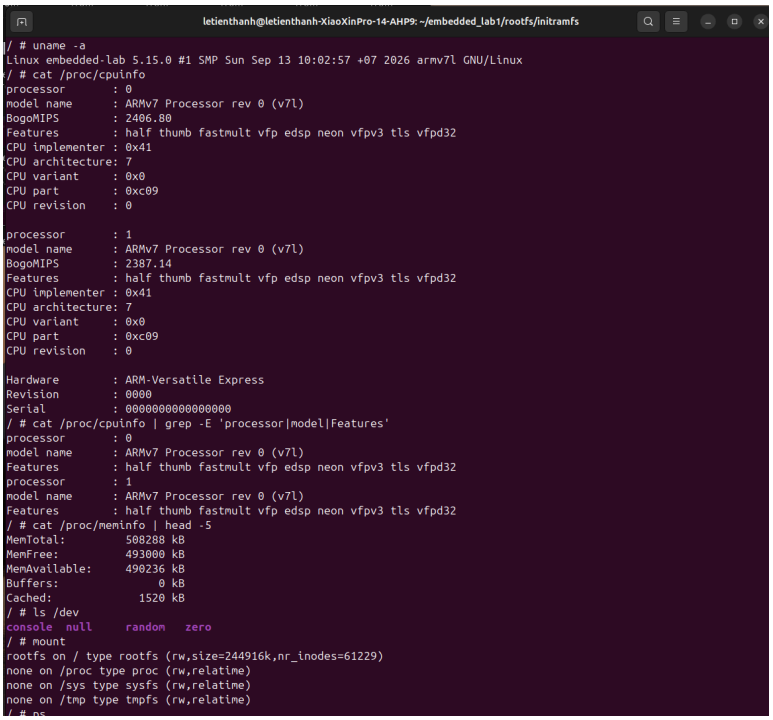
```
cd rootfs/initramfs
find . | cpio -H newc -o | gzip > ../../output/initramfs.cpio.gz
```

Không bắt buộc có shared libraries vì BusyBox đã link static. Với binary link động, cần dynamic loader và các thư viện được báo bởi readelf -d; thiếu loader có thể khiến init thất bại mặc dù file binary có tồn tại.

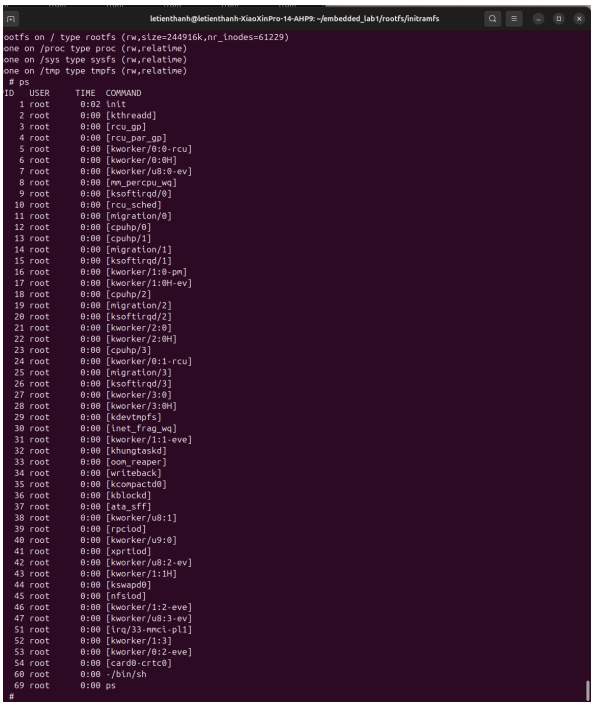
5.2. QEMU boot trực tiếp để cô lập Kernel/rootfs

```
qemu-system-arm -M vexpress-a9 -m 512M \
  -kernel output/zImage \
  -dtb output/vexpress-v2p-ca9.dtb \
  -initrd output/initramfs.cpio.gz \
  -append "console=ttyAMA0 rdinit=/sbin/init" -nographic
```

Boot trực tiếp thành công là phép kiểm tra độc lập cho Kernel, DTB, initramfs và rcS. Console xuất hiện thông báo “Boot completed!”, sau đó lệnh uname, mount, ps hoạt động. procfs, sysfs và tmpfs được mount; /proc/cmdline hiện đúng console=ttyAMA0 rdinit=/sbin/init.



Hình 8. Minh chứng của nhóm: uname, CPU info, thiết bị và các mountpoint trong BusyBox.



Hình 9. Danh sách tiến trình: BusyBox init tại PID 1 và shell tương tác.

5.3. Tổng hợp phép thử và giới hạn

Hạng mục	Quan sát	Trạng thái
zImage/DTB/initramfs từ bản clone	Đủ ba file và file zImage là ARM	PASS
U-Boot 2022.04	Khởi động tới =>	PASS
bootz + DTB + initramfs	Đạt Starting kernel và BusyBox / #	PASS
BusyBox/init + mount	PID 1 là init; proc/sysfs/tmpfs hoạt động	PASS

Hạng mục	Quan sát	Trạng thái
Số CPU	Lần boot tích hợp mới báo chỉ 1 CPU; lần khác đã chạy -smp 2	Phụ thuộc tham số
Báo cáo PDF có trên GitHub	Clone lần kiểm tra trước chưa có bao_cao/	Cần bổ sung

6. Trả lời câu hỏi phân tích

6.1. ABI soft-float và hard-float trên ARM

arm-linux-gnueabi và arm-linux-gnueabihf khác nhau trọng tâm ở cách truyền tham số số thực theo ABI: soft-float đưa đối số qua nhóm thanh ghi thông thường, còn hard-float dùng thanh ghi VFP khi giao tiếp hàm. Cortex-A9 hỗ trợ VFPv3/NEON, nên ABI hard-float phù hợp với userspace dùng thư viện armhf; tuy nhiên một chương trình gnueabi vẫn có thể phát sinh chỉ thị FPU nếu chọn softfp, không đồng nghĩa "không dùng được FPU". Kernel Linux thông thường không cho phép dùng float trực tiếp trong mã kernel và biên dịch theo quy ước riêng; ABI userspace không phải tiêu chí duy nhất quyết định Kernel có boot được.

6.2. Vì sao RAM bắt đầu tại địa chỉ 0x60000000?

Memory map của Versatile Express V2P-CA9 đặt RAM bắt đầu tại 0x60000000. Vùng địa chỉ thấp hơn chứa flash và peripheral, nên việc tải kernel/DTB không thể giả định RAM ở 0x00000000 như một số board ARM cũ. Bên cạnh ranh giới RAM, phải tránh ghi đè lẫn nhau và tránh vùng U-Boot đang dùng, vì điều này có thể làm hỏng FDT trước khi kernel chạy. Lần lỗi FDT_ERR_BADSTRUCTURE ở địa chỉ mẫu nhắc đến cả hai điều kiện này.

6.3. Giải mã CPU ID 0x410FC090

Trường bit	Giá trị	Diễn giải
Implementer [31:24]	0x41	ARM
Variant [23:20]	0x0	r0
Architecture [19:16]	0xF	Dùng hệ CPUID
Part [15:4]	0xC09	Cortex-A9
Revision [3:0]	0x0	p0

CPU ID này khớp boot log ARMv7 Processor [410fc090] và thông tin CPU part 0xc09 trong /proc/cpuinfo, xác nhận QEMU đang mô phỏng nhân Cortex-A9.

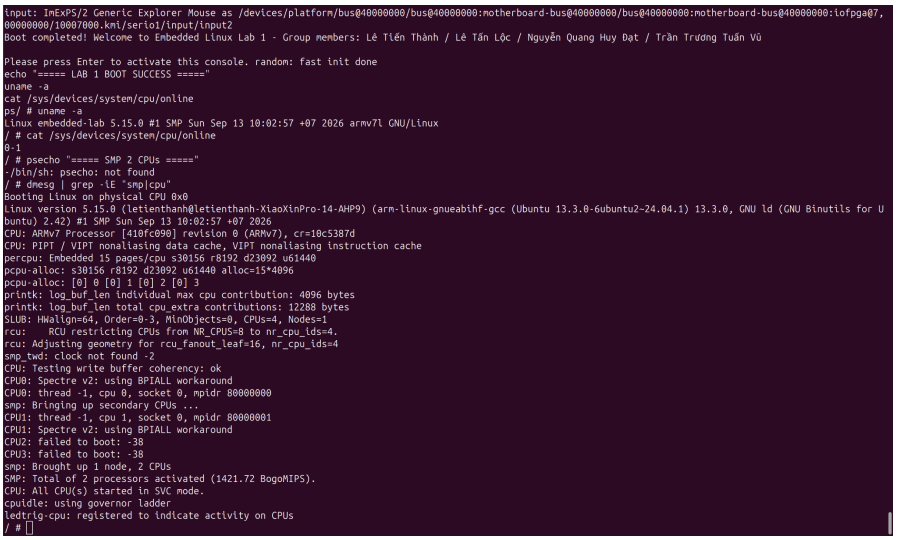
6.4. Lợi ích của BusyBox liên kết tĩnh

Một rootfs cực nhỏ không có dynamic linker và libc để khởi động binary động. Link tĩnh đóng gói các phụ thuộc cần thiết vào BusyBox, làm cho /sbin/init có thể chạy ngay sau khi Kernel giải nén initramfs. Đánh đổi là binary lớn hơn và mỗi ứng dụng phải mang mã thư viện nếu xây dựng nhiều chương trình. Khi chuyển sang liên kết động, cần xác định dependency thực tế qua readelf và sao chép đúng loader/thư viện cho ARM hard-float.

6.5. SMP, possible CPUs và online CPUs

SMP bổ sung cơ chế đánh thức CPU phụ, coherence, IPI, dữ liệu per-CPU và phân phối runnable task theo từng CPU. Khác biệt giữa "possible" và "online" rất quan trọng: possible=0-3 có thể xuất hiện theo cấu hình DTB/kernel dù QEMU chỉ cấp -smp 2. Số lõi đang chạy phải xác nhận bằng

/sys/devices/system/cpu/online hoặc dòng “SMP: Total of N processors activated”, không suy ra riêng từ tên kernel thread. Trong log boot qua U-Boot gần nhất kernel báo 1 CPU active; đây là kết quả thật của cấu hình lần đó, không phải hai hoặc bốn.



Hình 10. Minh chứng riêng của nhóm khi kiểm tra SMP ở một lần QEMU có cấu hình nhiều lõi.

7. Khó khăn, nhận xét và kết luận

Khó khăn thứ nhất là cấu hình menuconfig ban đầu nhầm sang host x86; nhóm phải đặt ARCH=arm trước khi xem các tùy chọn. Khó khăn thứ hai là các cờ CPU_V7/CPU_32v7 chưa xuất hiện ở lần kiểm tra đầu; việc chạy lại defconfig ARM giúp hệ thống nhận đúng cấu hình. Khó khăn thứ ba là hiểu sự khác nhau giữa CPU possible và CPU online. Lần test tích hợp còn cho thấy địa chỉ FDT dù nằm trong RAM vẫn có thể không phù hợp nếu chồng vào vùng bộ nhớ U-Boot xử lý.

Kết luận: các file nhân Linux, DTB, BusyBox và U-Boot đã chứng minh khởi động được trên bản clone GitHub; cả hai tuyến kiểm thử “QEMU boot trực tiếp” và “bootz qua U-Boot” đều có log đi đến shell. Để hoàn thiện gói nộp bài, nhóm cần bổ sung báo cáo PDF vào bao_cao/ và lưu lại kích thước initramfs thực dùng trong câu lệnh bootz cuối.

Phụ lục — Lệnh xác minh và nguồn đối chiếu

```
file output/zImage output/u-boot rootfs/initramfs/bin/busybox
git ls-files --stage embedded_lab1 | grep ^160000
stat -c %s output/initramfs.cpio.gz
# Trong BusyBox: uname -a; mount; ps; cat /proc/cmdline
```

Tư liệu kiểm chứng: ảnh và log thực hành LAB-01 của nhóm; log QEMU trên bản clone LOS201 ngày 21/10/2026. Tài liệu đối chiếu: đề LAB-01 môn Embedded Linux Systems; Lab01_Report.pdf (tham khảo hạng mục kiến thức). Sơ đồ do nhóm biên soạn lại theo kết quả quan sát.